

G.TECH

CAHIER DES CHARGES

S1P2 - PATHFINDING

Projet Hack'n Slash 3D

GTECH 2 - Binôme libre

Deux semaines de projet - 4 jours par semaine

Du 2 au 13 novembre 2026

Mini-moteur custom C++17 / raylib

Auteur: William Rao Fernandes

SÉANCES

SEMAINE 1 - CONSTRUIRE UNE TRANCHE JOUABLE

- **J1 :** cadrage, répartition des tâches, compilation et arène de test.
- **J2 :** intégration de Dijkstra, de A*, du NavMesh et des premiers tests.
- **J3 :** suivi de chemin, poursuite, attaque du joueur et premier ennemi.
- **J4 :** première code review, tranche jouable et corrections prioritaires.

SEMAINE 2 - COMPLÉTER ET FIABILISER LE JEU

- **J5 :** second comportement ennemi, vagues, victoire, défaite et redémarrage.
- **J6 :** deuxième code review, seconde scène, mesures et cas limites.
- **J7 :** recette complète, corrections finales, documentation et gel des fonctionnalités.
- **J8 :** livraison, démonstration et soutenance finale.

Organisation : huit journées de projet, à raison de quatre jours par semaine.

Chaque semaine fait l'objet d'une évaluation formative, fondée sur le travail réalisé, la progression, les tests et les corrections apportées après la revue.

Les deux semaines comprennent du développement et des code reviews. Le socle réalisé en théorie est réutilisé pour tenir le périmètre sur huit journées. Les bonus ne sont engagés qu'après validation des fonctionnalités obligatoires.

PROJET

MODALITÉS

Projet en **binôme libre** pour la classe GTECH 2. La semaine de théorie du 26 au 30 octobre constitue le préalable. Le développement se déroule du **2 au 13 novembre 2026**, à raison de **quatre jours par semaine, soit huit journées de projet**.

Le socle de cours et les composants écrits pendant les TP peuvent être réutilisés. Chaque membre doit comprendre et pouvoir modifier le pathfinding et les comportements ennemis. Les rôles de développement et de relecture alternent régulièrement.

DESCRIPTION

Le projet consiste à concevoir et développer un **Hack'n Slash 3D jouable en solo**, en vue de dessus ou légèrement inclinée. Le joueur évolue dans une arène comprenant des obstacles et des passages contraints. Il doit survivre à trois vagues et éliminer les ennemis de la dernière pour gagner.

Les ennemis doivent détecter le joueur selon des règles explicites, calculer un chemin pour le rejoindre, contourner les obstacles et attaquer lorsqu'ils sont à portée. Le jeu doit comporter deux archétypes dont les prises de décision sont distinctes et lisibles.

Une partie vise une durée de **trois à cinq minutes**. Le thème et l'univers visuel sont libres. Les primitives 3D et les assets simples sont acceptés : la priorité porte sur la justesse des algorithmes, la qualité de la navigation et la lisibilité du combat.

Le jeu est réalisé avec un **mini-moteur custom en C++17 construit sur raylib**. La bibliothèque fournit l'affichage et les entrées ; le binôme implémente ses systèmes de navigation, de décision et de gameplay. Aucun moteur généraliste ni agent de navigation prêt à l'emploi ne remplace ce travail.

RÉSULTAT ATTENDU

Une version jouable, testable et relançable, accompagnée du code source, des données de niveau et d'un rapport technique. Une démonstration doit rendre visibles les chemins calculés, les états ennemis et la comparaison entre Dijkstra et A*.

SPECS TECHNIQUES

1. Technologies utilisées

- **Moteur :** mini-moteur pédagogique custom. Aucune dépendance à Unreal, Unity, Godot ou à un autre moteur généraliste.
- **Langage et compilation :** C++17 et CMake. La version de raylib est figée dans le dépôt ; la référence des supports est raylib 5.5.
- **Navigation :** graphes pondérés, Dijkstra, A* et composant NavMesh développés par le binôme.
- **IA :** automate à états ou structure équivalente dont les conditions et transitions sont explicables.
- **Versionnement :** dépôt Git commun et historique des contributions. Les dépendances et assets sont accompagnés de leurs références et licences.

2. Contraintes de développement

Gameplay et structure

- Le joueur se déplace au clavier et dispose d'une attaque de mêlée orientée. Les commandes et les points de vie sont visibles.
- L'arène comprend au moins trois obstacles solides, un détour obligatoire et un passage étroit réellement praticable par les agents.
- Trois vagues sont jouables, dont une avec au moins dix ennemis simultanés. Les spawns sont valides et ne se trouvent ni dans les murs ni hors surface navigable.
- Le jeu gère la victoire, la défaite et le redémarrage sans fermer l'application. Une cible ne reçoit les dégâts qu'une fois par attaque ; les murs bloquent les attaques.

Architecture et optimisation

- Séparer les données du niveau, la recherche de chemin, le suivi du chemin, la décision, le mouvement, le combat et le debug.
- Conserver les paramètres de vie, vitesse, portée et délais dans des données ou des variables nommées. Les recherches ne sont pas déclenchées systématiquement à chaque frame.

SPECS TECHNIQUES

Pathfinding et NavMesh

- **Dijkstra et A* :** les deux algorithmes sont implémentés et comparables sur les mêmes graphes, coûts et couples départ/arrivée. Une bibliothèque de recherche de chemin ne peut pas les remplacer.
- **Coûts et heuristique :** les coûts sont non négatifs ; l'heuristique de A* est justifiée au regard des transitions. Les chemins trouvés ont le même coût optimal, à une tolérance de 10^4 près.
- **Surface :** le NavMesh comprend des polygones convexes et leurs adjacences par portails. Il représente les positions autorisées du centre pour un rayon d'agent fixé. Une grille seule ne valide pas cette exigence.
- **Requêtes :** localiser départ et arrivée, rechercher une suite de régions et produire des points de passage. Les segments doivent rester dans la surface ; la ligne directe entre centres ne doit pas couper un coin.
- **Échecs :** distinguer Found, NoPath, OutsideNavMesh et InvalidInput. Gérer départ égal à arrivée, zone déconnectée et données invalides sans crash ni traversée des murs.
- **Réutilisation :** charger deux scènes décrites par des données distinctes, sans modifier les algorithmes. Le NavMesh manuel sur un sol XZ horizontal est suffisant ; le bake automatique et les étages ne sont pas exigés.

IA ennemie et combat

- Chaque ennemi possède au minimum des comportements d'attente, de détection, de poursuite, de préparation d'attaque, de récupération et de mort. Une perte de cible ou un échec de chemin entraîne une réaction définie.
- Deux archétypes sont requis : un poursuivant léger et un combattant lourd. Le lourd s'immobilise pendant sa préparation annoncée ; le léger conserve une approche mobile ou applique une autre règle de décision distincte. Changer uniquement les statistiques ne suffit pas.
- À l'impact, revérifier portée et absence de mur. Un ennemi mort ne se déplace plus et ne peut plus infliger de dégâts.
- Le suivi tient compte du temps écoulé et borne l'avance aux waypoints. Les recalculs sont temporisés. Une séparation simple ou des positions d'attaque réparties limitent les superpositions ; les limites en passage étroit sont documentées.

SPECS TECHNIQUES

Graphisme et feedbacks

- Le rendu doit permettre de distinguer joueur, ennemis légers, ennemis lourds, obstacles et limites de l'arène. Les objets restent lisibles avec des formes simples.
- Un feedback visuel indique au minimum une attaque en préparation, un coup reçu, une mort et la fin de partie. Les shaders personnalisés et effets complexes sont facultatifs.

Audio

- Prévoir au moins un son d'attaque et un son de dégâts ou de mort. Les sons servent la compréhension du combat et leurs niveaux ne masquent pas les autres informations. Une ambiance ou une musique reste facultative.

Debug et performances

- Une commande affiche ou masque NavMesh, portails, chemin et état de l'agent sélectionné. Le debug indique l'algorithme, le statut, le coût, les sommets développés et le temps de recherche.
- Comparer dix requêtes fixées, répétées cent fois en Release, sans rendu dans la mesure. Rapporter médiane et p95 ; préciser la machine et ce qui est chronométré.
- Cible indicative : 60 images/s à 1280 × 720 avec dix ennemis sur un poste du campus identifié au lancement. Mesurer aussi le temps de frame médian et p95 pendant 60 secondes. Tout écart est analysé ; ce seuil seul n'est pas éliminatoire.

Fichiers et organisation

- Organisation minimale : **/src/Navigation/**, **/src/AI/**, **/src/Gameplay/**, **/assets/**, **/data/levels/**, **/tests/** et **/docs/**. Les fichiers et ressources portent des noms explicites.
- Le projet compile depuis une copie propre du dépôt sur la cible du campus. Aucun chemin absolu propre au poste auteur ne doit être nécessaire.

Périmètre : réseau, inventaire complexe, loot avancé, génération procédurale, animation squelettique et physique complète ne sont pas requis. Un bonus de navigation est possible après validation du socle ; il ne compense pas une exigence manquante.

LIVRABLES ET VALIDATION

Livraison du vendredi 13 novembre 2026

- **Code source :** dépôt Git avec tag final, CMake, version des dépendances, données des deux niveaux et tests exécutables.
- **Version jouable :** build pour le poste cible, assets et README indiquant installation, compilation, commandes, debug et limites connues.
- **Note technique de 3 à 5 pages :** architecture, coûts et heuristique, NavMesh et rayon, automate, mesures, anomalies et choix de périmètre.
- **Preuves de validation :** résultats de recette, données brutes des mesures, journal des deux code reviews, corrections et contributions des membres.

Cas de recette obligatoires

Les cas ci-dessous doivent être rejouables. Les tests de calcul appellent le code de production ; les scénarios de jeu indiquent les étapes, le résultat attendu et le résultat réel.

Cas	Résultat attendu
Graphe pondéré et requêtes comparées	Même coût optimal pour Dijkstra et A*, même statut en cas d'échec.
Départ égal à arrivée	Succès, coût nul et aucun déplacement inutile.
Îlot inaccessible ou point extérieur	Statut approprié ; aucun mouvement à travers un mur.
Entrée invalide ou coût négatif	Rejet explicite et aucun crash.
Couloir en L et passage étroit	Portails suivis ; volume de l'agent compatible avec le passage.
Cible mobile et mur à portée	Recalcul maîtrisé ; aucun dégât à travers le mur.
Dix ennemis et partie complète	Stabilité, trois vagues, victoire, défaite et reset corrects.

La livraison est recevable pour démonstration si le build démarre, qu'une partie peut se terminer et que la navigation est observable. Les échecs restent consignés et affectent les critères concernés du barème ; ils ne sont pas masqués dans le rapport.

OBJECTIFS PÉDAGOGIQUES

À l'issue du projet, les étudiants doivent pouvoir démontrer les connaissances, compétences et comportements suivants.

SAVOIR - Connaissances à acquérir

- Expliquer le rôle des sommets, arêtes et poids dans la représentation d'un espace navigable.
- Distinguer Dijkstra et A*, définir une heuristique admissible et cohérente, et justifier le coût optimisé.
- Expliquer la différence entre surface navigable, trajet de régions, trajet géométrique et collision du corps de l'agent.
- Distinguer perception, décision, navigation et mouvement ; différencier IA gameplay et IA générative.

SAVOIR-FAIRE - Compétences techniques

- Implémenter et tester les deux recherches sur une représentation commune, en traitant les cas limites.
- Construire un NavMesh polygonal, ses portails et un suivi de chemin compatible avec un rayon fixé.
- Concevoir deux comportements ennemis distincts et les intégrer à une boucle de combat complète.
- Instrumenter, mesurer et corriger la navigation ; utiliser le debug pour expliquer une anomalie.
- Compiler, livrer, documenter et présenter une version jouable ; corriger le code après relecture.

SAVOIR-ÊTRE - Attitudes professionnelles

- Travailler avec rigueur, analyse logique et persévérance face aux erreurs de calcul ou de comportement.
- Répartir les tâches, alterner les rôles et relire le travail de son binôme avec des retours précis.
- Prioriser le socle jouable, respecter le délai et expliquer les compromis retenus.

Acculturation IA : documenter une expérience avec 1, 3 puis 10 agents, répétée trois fois. Décrire le comportement collectif observé, la règle locale qui peut l'expliquer et les limites de cette interprétation. Les usages significatifs d'un assistant génératif sont déclarés ; tout code intégré doit rester explicable.

GRILLE D'ÉVALUATION

Le barème reprend une évaluation sur **60 points**. La note sur 20 est obtenue en divisant le total par 3. Les retours formatifs des deux semaines préparent cette évaluation finale.

CRITÈRES	POINTS
FONDAMENTAUX ET NAVIGATION	
Architecture, graphe et cohérence des coûts	/ 6
Dijkstra : exactitude et reconstruction	/ 6
A* : exactitude et justification de l'heuristique	/ 6
NavMesh, portails, rayon et suivi de chemin	/ 12
IA ET DÉVELOPPEMENT	
Deux comportements ennemis, transitions et combat	/ 6
Boucle de jeu, vagues et lisibilité du gameplay	/ 4
Debug, mesures et maîtrise des recalculs	/ 4
PROJET APPLIQUÉ	
Tests, documentation, collaboration et corrections	/ 6
Rendu final et stabilité de la livraison	/ 4
Soutenance et analyse critique de l'IA	/ 6
TOTAL	/ 60

Soutenance finale - 20 minutes

5 min : démonstration de la boucle de jeu.
5 min : comparaison Dijkstra / A* et visualisation du NavMesh.
5 min : explication d'une fonction et d'un cas limite par les deux membres.
5 min : mesures, corrections et analyse des comportements observés.

La note de groupe est étayée par le code, les tests et la démonstration. Une maîtrise individuelle différente peut conduire à individualiser les critères techniques sur la base des explications et d'une courte modification en séance. Les bonus enrichissent les critères existants sans dépasser 60 points.